

Laboratory 2 Report

ANDREA BOTTICELLA*, s347291, Politecnico di Torino, Italy

ELIA INNOCENTI*, s345388, Politecnico di Torino, Italy

SIMONE ROMANO*, s344024, Politecnico di Torino, Italy

This work addresses malware detection through behavioral analysis of API call sequences. We implement and compare four classification approaches: a frequency-based logistic regression baseline, Feed-Forward Neural Networks with learned embeddings, Recurrent Neural Networks (RNN, BiRNN, LSTM) for sequential modeling, and Graph Neural Networks (GCN, GraphSAGE, GAT) that represent call sequences as transition graphs. Experiments are conducted on a dataset of execution traces containing 258 unique API calls, with attention to class imbalance handling and variable-length sequence processing.

1 INTRODUCTION

Dynamic malware analysis leverages runtime behavior to detect malicious software, circumventing the limitations of static signature-based methods against obfuscated or polymorphic variants. API call sequences—the ordered invocations a program makes to the operating system—provide a rich behavioral fingerprint, capturing memory operations, registry accesses, and system queries that reveal malicious intent.

This laboratory implements a comparative study of neural network architectures for binary malware classification based on API call sequences. The dataset consists of execution traces, each represented as a variable-length sequence of API function names (e.g., `NtAllocateVirtualMemory`, `RegOpenKeyExW`).

We evaluate four progressively complex approaches:

- **1:** Implement and evaluate a frequency-based logistic regression baseline.
- **2:** Develop Feed-Forward Neural Networks with different input representations.
- **3:** Explore Recurrent Neural Network architectures for sequential modeling.
- **4:** Construct Graph Neural Networks to leverage structural information in API call sequences.

The following sections detail the methodology and results for each task, analyzing how architectural choices impact classification performance.

2 TASK 1: FREQUENCY-BASED BASELINE

This section establishes a baseline for malware classification using a frequency-based representation of API call sequences. The intent is to quantify how far one can go with a simple model that ignores temporal order, and to provide a reference point for the more expressive architectures introduced later.

2.1 Data Loading and Exploration

The dataset is provided in JSON format, with each sample containing two fields: `api_call_sequence` (a list of API function names observed during execution) and `is_malware` (a binary label). Both training and test partitions are loaded for an initial inspection of size, label distribution, and vocabulary.

The training set contains 16,325 samples, while the test set comprises 6,505 samples. A striking observation emerges from examining the class distribution: malware samples constitute 96.26% of both partitions (15,714 training and 6,262 test samples), leaving only 3.74% for benign software (611 training and 243 test samples). This severe class imbalance has important implications for model evaluation. Accuracy alone would be misleading, as a trivial classifier predicting “malware” for every sample would achieve 96% accuracy without learning anything meaningful. We therefore pay close attention to precision, recall, and F1-score for both classes, particularly monitoring performance on the minority benign class.

2.2 Vocabulary Extraction and Analysis

The frequency-based representation requires a fixed vocabulary. To avoid any information leakage, the vocabulary is extracted from the training set and used to define the feature space at inference time. The training set contains 258 unique API calls, while the test set contains 232.

*The authors collaborated closely in developing this project.

This immediately exposes a vocabulary mismatch. Twenty-nine API calls appear only in the training set (they can influence training but never occur at test time), while three appear exclusively in the test set: `NtDeleteKey`, `ControlService`, and `WSASocketA`. The latter case is the critical one, because the classifier would be asked to process signals that were not available during training.

Unseen API calls can be handled by introducing an explicit unknown feature (e.g., `<UNK>`) or by discarding them when building the input vector. Since the three unseen calls appear in only three test samples, they have negligible impact at the dataset scale; the pipeline therefore ignores them and builds test vectors strictly over the training vocabulary.

2.3 Frequency-Based Feature Vectors

Each API call sequence is transformed into a 258-dimensional vector, where each element is the count of occurrences of a specific API call from the training vocabulary. This bag-of-words representation discards temporal ordering: any permutation of the same multiset of calls maps to the same vector, and patterns that depend on call order are not represented.

The resulting feature matrices are highly sparse, as individual execution traces invoke only a small subset of the full API vocabulary. On average, training samples contain 21.95 non-zero elements per row, representing just 8.51% of the 258 dimensions. Test samples show slightly higher density with 24.28 non-zero elements (10.46%). This sparsity has practical implications: memory-efficient sparse matrix representations can be leveraged, and classifiers designed for high-dimensional sparse data become particularly suitable choices.

2.4 Classifier Selection and Training

Logistic regression is adopted as the baseline classifier for its computational efficiency on sparse high-dimensional inputs and its transparency: the learned coefficients provide a direct indication of which API calls are associated with either class. The `liblinear` solver is selected for its robustness in this regime, while hyperparameters are left at default values to preserve the role of this model as a minimal, reproducible baseline.

A baseline is not a formality. It anchors the comparison with the more complex architectures in the following tasks and prevents attributing improvements to architectural sophistication when the same gains could be achieved by a simpler representation or by exploiting label imbalance.

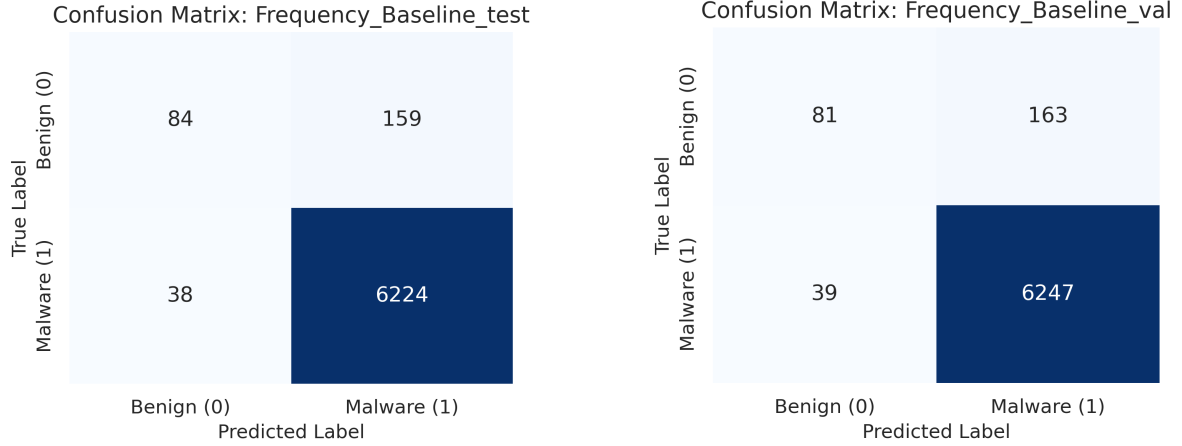
The data is split into 60% training and 40% validation using stratified sampling to preserve class proportions. Training produces a log-loss of 0.0806 on the training set and 0.1225 on the validation set. The gap between these values suggests mild overfitting, but remains acceptable for a baseline model without regularization tuning.

2.5 Evaluation and Results

We evaluate the trained classifier on both validation and held-out test sets. Table 1 presents the full classification report for the test set.

Table 1. Frequency-based baseline: classification report (test set)

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.6885	0.3457	0.4603	243
Malware (1)	0.9751	0.9939	0.9844	6,262
Accuracy		0.9697		6,505
Macro avg	0.8318	0.6698	0.7223	6,505
Weighted avg	0.9644	0.9697	0.9648	6,505



(a) Confusion matrix for the frequency-based baseline on the test set.

(b) Confusion matrix for the frequency-based baseline on the validation set.

Fig. 1. Comparison of confusion matrices for the frequency-based baseline on test and validation sets.

The results reveal a markedly asymmetric behavior driven by the underlying imbalance. Malware detection is extremely strong (precision 97.51%, recall 99.39%, F1-score 0.9844), and the confusion matrices in Figure 1 confirm that false negatives are rare. The benign class is more challenging: although benign predictions are often correct (precision 68.85%), recall is low (34.57%), meaning that a large fraction of benign samples are flagged as malware. With only 3.74% benign samples, the model learns a conservative decision boundary that favors catching malware at the cost of additional false alarms.

Despite ignoring order and operating on sparse vectors, the frequency-based baseline reaches 96.97% accuracy and near-perfect malware recall. This shows that the presence and relative frequency of specific API calls already carries substantial discriminative signal, even before exploiting sequential dependencies.

The limitation is structural: frequency vectors cannot represent temporal motifs, repeated call patterns, or long-range dependencies. This motivates the sequence- and structure-aware models in the subsequent tasks, where the representation is designed to preserve more of the behavioral dynamics that are erased by the bag-of-words transformation.

3 TASK 2: FEED FORWARD NEURAL NETWORK (FFNN)

The Feed-Forward Neural Network represents the first neural architecture applied to the malware detection task and serves as a controlled transition from feature-based baselines to representation learning. Its adoption immediately exposes two structural constraints of FFNNs: the requirement of fixed-size inputs and the inability to explicitly model temporal dependencies. The entire design of this task is therefore shaped by the need to adapt sequential, variable-length, categorical data to a static vector-based model.

3.1 Sequence Length Variability and Fixed-Size Encoding

API call sequences are inherently variable in length. A preliminary statistical analysis shows that neither the training nor the test set exhibits uniform sequence sizes. Moreover, the two distributions are not aligned: test sequences are, on average, longer and shifted toward higher values. This mismatch is illustrated in Figure 2.

Since an FFNN operates on fixed-dimensional vectors, each sequence must be transformed to a common length. The target size was estimated exclusively on the training set to avoid information leakage. Among the evaluated candidates, the maximum observed training length was selected, ensuring that no training information was discarded. Sequences shorter than this threshold were padded using a dedicated token, while longer sequences were truncated at the tail. The same transformation was applied to the test set to preserve dimensional consistency at inference time.

This operation enforces a uniform representation but introduces an unavoidable trade-off: padding increases input sparsity, while truncation removes potentially informative late-stage API calls. These limitations are intrinsic to FFNN-based modeling of sequences and motivate the comparison with more expressive architectures in later tasks.

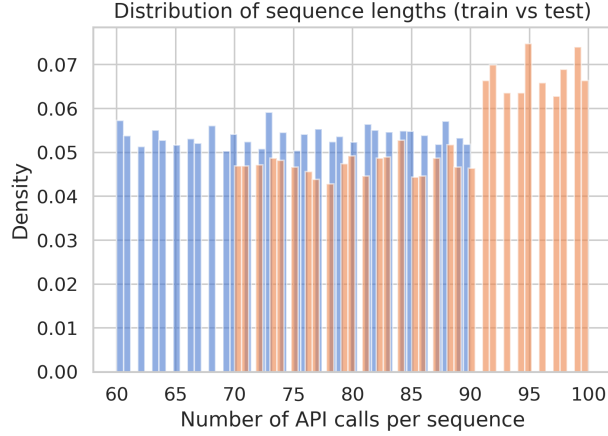


Fig. 2. Distribution of API call sequence lengths in training and test sets.

3.2 Categorical Representation of API Calls

API calls are symbolic entities and must be mapped to a numerical space. A unified vocabulary was built from the padded training sequences, augmented with two special symbols: a padding token and an unknown token, the latter used to handle API calls appearing only in the test set.

Two alternative encoding strategies were considered.

Sequential Identifiers. Each API call is mapped to an integer identifier. This approach is simple and compact but introduces no semantic structure: numerical proximity between identifiers is arbitrary and does not reflect behavioral similarity. The FFNN must therefore infer all relationships implicitly from co-occurrence patterns across fixed positions.

Learnable Embeddings. In the second configuration, integer identifiers are projected into a dense vector space through a learnable embedding layer. This mapping is optimized jointly with the classifier and allows the model to encode latent similarities between API calls that frequently appear in related contexts. Compared to raw identifiers, embeddings provide a smoother and more expressive input space, at the cost of additional parameters.

3.3 FFNN Architecture

The FFNN architecture consists of an optional embedding layer followed by a stack of fully connected layers with decreasing width. ReLU activations are used throughout, with batch normalization and dropout applied to stabilize training and reduce overfitting. When embeddings are enabled, the embedded sequence is flattened into a single vector before being processed by the feed-forward layers. The output layer produces a single logit for binary classification.

3.4 Learning Rate Sensitivity Analysis

To assess training stability and convergence behavior, multiple learning rates were evaluated for each architecture. Figure 3 compares training and validation loss curves across learning rates for the sequential identifier configuration.

Higher learning rates accelerate early convergence but lead to noisier validation curves and less stable plateaus. Lower learning rates produce smoother trajectories but require more epochs to reach comparable performance. An intermediate value yields the best trade-off between convergence speed and stability, minimizing validation loss without oscillatory behavior.

A complementary view is provided in Figure 4, where training and validation losses are shown side by side for each learning rate.

These plots reveal that excessively large learning rates push the model toward premature convergence with limited generalization, while excessively small values slow optimization without clear performance gains. The selected configuration corresponds to the lowest validation loss with consistent train-validation alignment.

FFNN Sequential IDs - Learning Rate Comparison

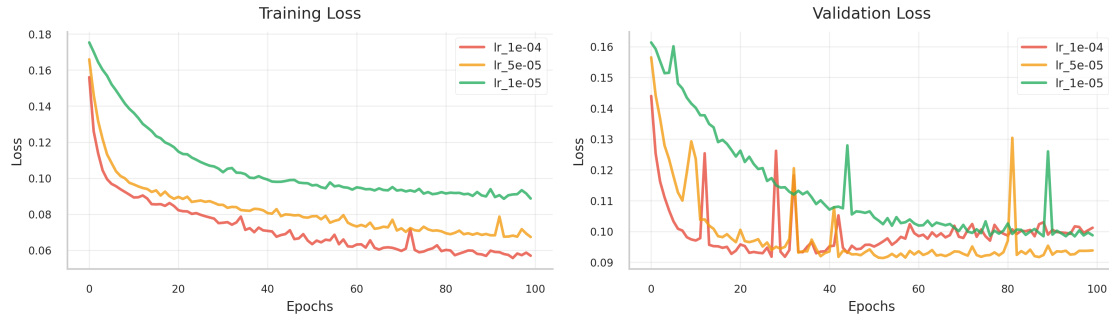


Fig. 3. Training and validation loss curves for the FFNN with sequential identifiers under different learning rates.

FFNN Sequential IDs - Overfitting Diagnosis (Train vs Val)

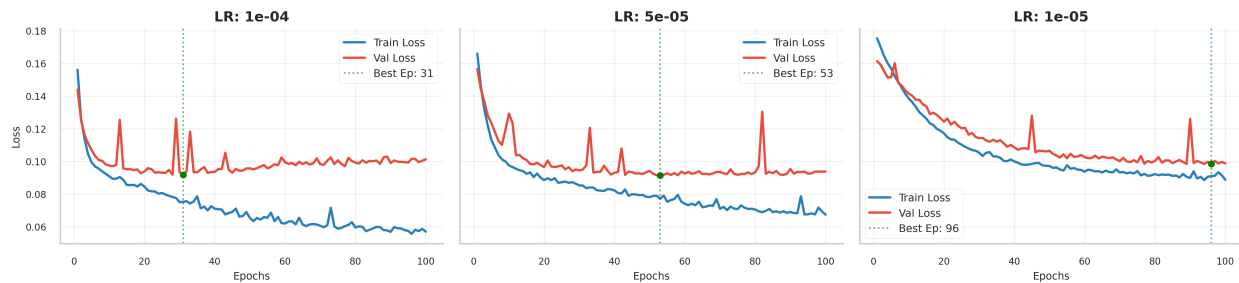


Fig. 4. Train vs. validation loss for different learning rates, highlighting overfitting and underfitting regimes.

3.5 Confusion Matrix Analysis

To complement scalar metrics, confusion matrices were analyzed on the test set. Figure ?? reports the confusion matrix for the best-performing FFNN using sequential identifiers.

Confusion Matrix: FFNN Sequential IDs (lr=1e-04)

True Label	Predicted Label	
	Benign (0)	Malware (1)
Benign (0)	62	181
Malware (1)	535	5727

Fig. 5. Learning rate = 1e-4

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)

True Label	Predicted Label	
	Benign (0)	Malware (1)
Benign (0)	17	226
Malware (1)	119	6143

Fig. 6. Learning rate = 1e-5

Confusion Matrix: FFNN Sequential IDs (lr=5e-05)

True Label	Predicted Label	
	Benign (0)	Malware (1)
Benign (0)	67	176
Malware (1)	591	5671

Fig. 7. Learning rate = 5e-5

The matrix highlights a strong bias toward the majority class. While malware samples are detected with high recall, benign samples are frequently misclassified. This behavior is consistent with the class imbalance of the dataset and confirms that overall accuracy alone is not a sufficient indicator of model quality. The FFNN effectively learns dominant patterns but struggles to establish robust decision boundaries for the minority class.

3.6 Learnable Embeddings: Architecture and Training Behavior

In parallel with the sequential identifiers configuration, the FFNN was trained using a learnable embedding layer as the first stage of the model. In this setting, each API call identifier is projected into a dense, low-dimensional vector space before being processed by the fully connected layers. The embedding matrix is optimized jointly with the classifier, allowing the model to learn distributed representations that capture latent similarities between API calls.

From an architectural standpoint, the introduction of embeddings changes the effective input geometry of the network. Instead of operating directly on discrete identifiers, the FFNN receives a continuous representation in which semantically related API calls can occupy nearby regions of the embedding space. After embedding, the sequence is flattened, preserving positional information but enriching each position with a learned feature vector rather than a scalar identifier.

3.7 Learning Rate Analysis for the Embedding-Based Model

As with the sequential identifiers approach, multiple learning rates were evaluated to assess optimization stability and convergence dynamics. Figure 8 reports the training and validation loss curves for the embedding-based FFNN under different learning rate values.

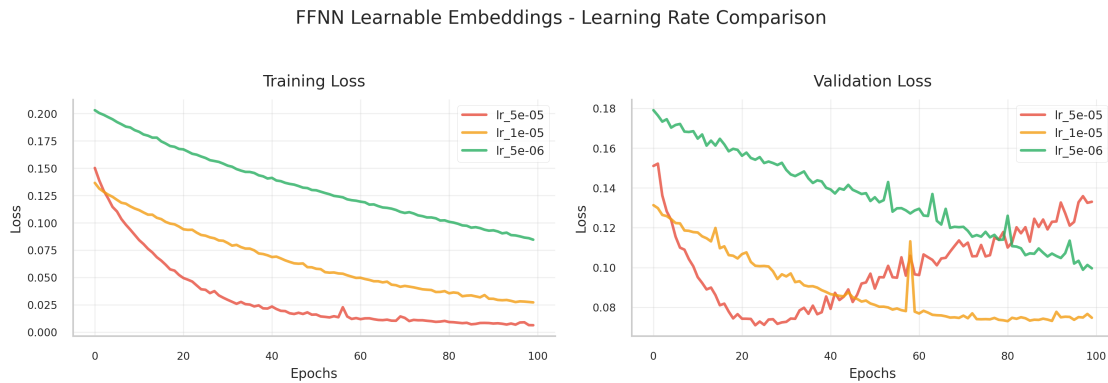


Fig. 8. Training and validation loss curves for the FFNN with learnable embeddings under different learning rates.

Compared to the sequential identifiers model, the embedding-based architecture exhibits smoother optimization trajectories and reduced sensitivity to the learning rate choice. Larger learning rates still accelerate early convergence, but oscillations in validation loss are attenuated, suggesting that the continuous embedding space stabilizes gradient updates. Lower learning rates converge more slowly but remain consistently aligned between training and validation losses.

A more detailed diagnosis is shown in Figure 9, where training and validation losses are displayed jointly for each learning rate.

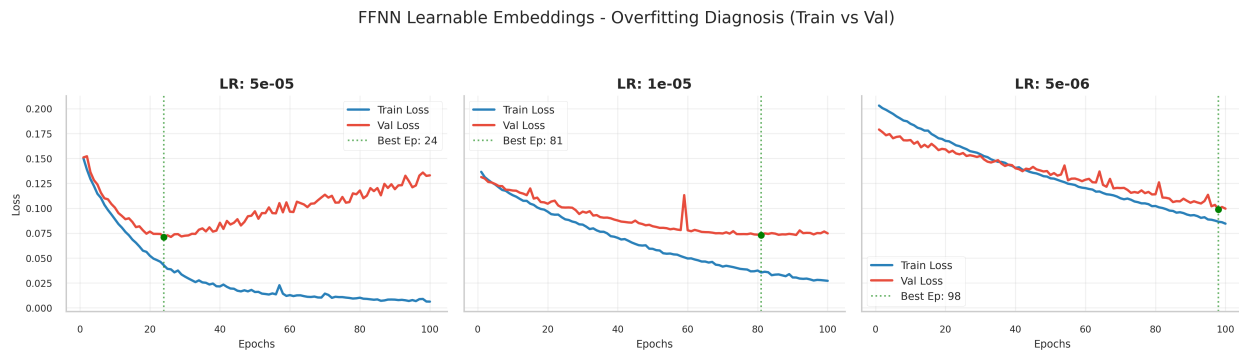


Fig. 9. Train vs. validation loss for the embedding-based FFNN, highlighting convergence behavior across learning rates.

Across all tested configurations, the embedding-based model shows reduced overfitting tendencies compared to the sequential identifiers variant. The gap between training and validation loss remains narrower, particularly in later epochs, indicating improved generalization.

3.8 Confusion Matrix Analysis: Embeddings vs. Sequential Identifiers

The impact of embeddings is further clarified through confusion matrix analysis. Figure ?? reports the confusion matrices obtained for the embedding-based FFNN under different learning rates.

Confusion Matrix: FFNN Learnable Embeddings ($\text{lr}=1\text{e-}05$)

True Label	Predicted Label	
	Benign (0)	Malware (1)
Benign (0)	54	189
Malware (1)	203	6059

Fig. 10. Learning rate = $1\text{e-}5$

Confusion Matrix: FFNN Learnable Embeddings ($\text{lr}=5\text{e-}05$)

True Label	Predicted Label	
	Benign (0)	Malware (1)
Benign (0)	46	197
Malware (1)	251	6011

Fig. 11. Learning rate = $5\text{e-}5$

Confusion Matrix: FFNN Learnable Embeddings ($\text{lr}=5\text{e-}06$)

True Label	Predicted Label	
	Benign (0)	Malware (1)
Benign (0)	94	149
Malware (1)	529	5733

Fig. 12. Learning rate = $5\text{e-}6$

Relative to the sequential identifiers model, embeddings reduce the asymmetry between classes. While the dataset remains heavily imbalanced and the majority class still dominates predictions, the embedding-based FFNN consistently improves recall and precision for the minority class. This effect is particularly visible in the reduction of false negatives for benign samples, indicating that the model benefits from a more expressive input representation.

3.9 Comparative Discussion

The comparison between sequential identifiers and learnable embeddings highlights a clear qualitative difference in how the FFNN internalizes API call information. Sequential identifiers force the network to treat API calls as isolated symbols whose relationships must be inferred indirectly through position-specific patterns. This results in brittle decision boundaries and strong bias toward the majority class, as evidenced by the confusion matrices and macro-averaged metrics.

In contrast, learnable embeddings provide a structured, continuous representation that allows the model to capture similarities between API calls that frequently co-occur in related behaviors. This leads to more stable training dynamics, reduced sensitivity to hyperparameter choices, and improved generalization on the minority class, even though temporal structure is still flattened.

Despite these improvements, both approaches share the same fundamental limitation: the sequential nature of API calls is encoded only implicitly through position, not through recurrence or state. The embedding-based FFNN therefore represents an incremental but meaningful step forward, setting a stronger baseline and justifying the transition to architectures that can explicitly model temporal dependencies in subsequent tasks.

4 TASK 3: RECURSIVE NEURAL NETWORK (RNN)

5 TASK 4: GRAPH NEURAL NETWORK (GNN)

6 CONCLUSIONS